

```
<!DOCTYPE html>

<html>

<head>

  <meta charset="UTF-8">

  <title>Bài toán tính toán</title>

</head>

<body>


<form action="" name="F">
  x <input type="text" name="y1">
  y <input type="text" name="y2"><br>
  Phép toán:
  <input type="radio" name="op" value="+"> +
  <input type="radio" name="op" value="-"> - <br>
  Kết quả: <input type="text" name="ketqua">
  <input type="button" onclick="tinhToan()" value="Kết Quả"><br>
  <div id="Err"></div>
</form>


<script>

function tinhToan() {
  // Lấy các giá trị đầu vào
  let x = document.F.y1.value.trim();
  let y = document.F.y2.value.trim();
  if (!isPrime(x) || !isPrime(y)) {
    alert("Vui lòng nhập các số nguyên tố.");
    return;
  }

  let ops = document.F.op;
```

```
let ketqua = document.F.ketqua;
let err = document.getElementById("Err");

// Xóa thông báo lỗi cũ
err.innerHTML = "";
err.style.color = "black";
err.style.fontWeight = "normal";

// Kiểm tra x, y có phải là số không
if (x === "" || y === "" || isNaN(x) || isNaN(y)) {
    err.innerHTML = "Vui lòng nhập số hợp lệ cho x và y.";
    err.style.color = "red";
    err.style.fontWeight = "bold";
    ketqua.value = "";
    return;
}

x = parseFloat(x);
y = parseFloat(y);

// Lấy phép toán được chọn
let toanTu = null;
if (ops.length === undefined) {
    if (ops.checked) {
        toanTu = ops.value;
    }
} else {
    for (let i = 0; i < ops.length; i++) {
        if (ops[i].checked) {
            toanTu = ops[i].value;
        }
    }
}
```

```
        break;
    }
}
}
```

```
// Nếu không chọn phép toán
if (toanTu === null) {
    err.innerHTML = "Vui lòng chọn phép toán.";
    err.style.color = "red";
    err.style.fontWeight = "bold";
    ketqua.value = "";
    return;
}
```

```
// Tính kết quả
let kq = 0;
if (toanTu === "+") {
    kq = x + y;
} else if (toanTu === "-") {
    kq = x - y;
}
```

```
// Gán kết quả và xóa lỗi nếu có
ketqua.value = kq;
err.innerHTML = "";
}
</script>

</body>
</html>
```

```

function isPrime(n) {
  if (n < 2 || n % 1 !== 0) return false;
  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i === 0) return false;
  }
  return true;
}

function isPerfectSquare(n) {
  if (n < 0) return false;
  let sqrt = Math.sqrt(n);
  return sqrt === Math.floor(sqrt);
}

function isPerfectNumber(n) {
  if (n <= 0 || n % 1 !== 0) return false;
  let sum = 0;
  for (let i = 1; i <= n / 2; i++) {
    if (n % i === 0) sum += i;
  }
  return sum === n;
}

function isPositiveInteger(n) {
  return Number.isInteger(n) && n > 0;
}

function isFibonacci(n) { if (n < 0) return false; let isSquare = x => { let s = Math.sqrt(x);
return s === Math.floor(s); }; return isSquare(5 * n * n + 4) || isSquare(5 * n * n - 4); }

let num = parseInt(document.F.y1.value);

if (isPrime(num)) console.log("Là số nguyên tố");
if (isPerfectSquare(num)) console.log("Là số chính phương");

```

```
if (isPerfectNumber(num)) console.log("Là số hoàn hảo");  
if (isPositiveInteger(num)) console.log("Là số nguyên dương");  
if (isFibonacci(num)) console.log("Là số Fibonacci");
```

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8" />  
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>  
  <title>Responsive Layout</title>  
  <style>  
    * { box-sizing: border-box; margin: 0; padding: 0; }  
  
    .container {  
      padding: 10px;  
      display: flex;  
      flex-direction: column;  
      gap: 10px;  
      max-width: 1024px;  
      margin: auto;  
    }  
  
    .box { height: 60px; background: gray; }  
  
    .orange { background: #f4a261; }  
    .yellow { background: #e9c46a; height: 120px; }  
    .blue { background: #2a9d8f; }  
    .gray { background: #d3d3d3; }
```

```
.grid { display: flex; flex-direction: column; gap: 10px; }
```

```
@media (min-width: 321px) {  
  .grid-gray {  
    display: grid;  
    grid-template-columns: repeat(2, 1fr);  
    gap: 10px;  
  }  
  .grid-gray .box { display: block; }  
}
```

```
@media (min-width: 769px) {  
  .grid-orange {  
    display: grid;  
    grid-template-columns: repeat(2, 1fr);  
    gap: 10px;  
  }  
  .grid-blue {  
    display: grid;  
    grid-template-columns: repeat(3, 1fr);  
    gap: 10px;  
  }  
  .grid-gray {  
    grid-template-columns: repeat(4, 1fr);  
  }  
}
```

```
@media (max-width: 320px) {  
  .grid-gray .box:nth-child(n+2) {  
    display: none;  
  }  
}
```

```
    }  
  }  
</style>  
</head>  
<body>  
  
  <div class="container">  
    <div class="grid grid-orange">  
      <div class="box orange"></div>  
      <div class="box orange"></div>  
    </div>  
  
    <div class="box yellow"></div>  
  
    <div class="grid grid-blue">  
      <div class="box blue"></div>  
      <div class="box blue"></div>  
      <div class="box blue"></div>  
    </div>  
  
    <div class="grid grid-gray">  
      <div class="box gray"></div>  
      <div class="box gray"></div>  
      <div class="box gray"></div>  
      <div class="box gray"></div>  
    </div>  
  </div>  
  
</body>  
</html>
```

